DeadlineDesk: A Campus Web System for Placement Rounds and Academic Deadlines

Project Proposal for UCS503P
Submitted to: Mr. Jeelani Asif
Thapar Institute of Engineering and Technology

Aryan Gupta, CSED* Aksh Goyal, CSED† Naveen Bansal, CSED‡

18 August 2026

Abstract

Placement application windows and assignment cutoffs at TIET are often communicated through WhatsApp forwards, shared drives, and scattered LMS notices. Students miss deadlines; teaching assistants apply late rules inconsistently. This proposal presents **DeadlineDesk**, a role-based web system that combines a placement track (companies, rounds, document checklists, reminders) with an academic dropbox (submission, late policy, similarity stub, TA grading). Success is measured by on-time completion and correct late-policy behaviour. The scope baseline covers work completed through the Week 7 prototype.

1 Introduction / Problem Statement

Campus academic and placement calendars place concurrent pressure on students. Placement cells publish company-specific application windows and document requirements; departments publish assignment due times with late policies. In practice, these obligations travel through informal channels: forwarded messages, spreadsheets, and drive folders. Context is lost, reminders are uneven, and last-minute rushes are common.

The gap is the lack of a single, enforceable system for two related deadline workflows. On the placement side, students rarely have a structured checklist and reminder trail for each round. On the academic side, submissions may bypass the LMS; late acceptance is decided manually; grading status is hard to track. Nearby course projects do not address this: CaseRoom focuses on interview practice and scoring; College Mentorship focuses on long-term mentor matching; generic campus-resource tools do not model round checklists or late-policy submission states.

DeadlineDesk closes the gap with a deployable web product: shared authentication and roles; a Placement Track for companies, rounds, document checklists, and reminders; and an Academic Dropbox for assignments, file upload, automated late evaluation, a lightweight similarity check stub, and TA grading. The problem is locally relevant, actionable with mature web frameworks, and suitable for measurable evaluation within one semester, consistent with UCS503P expectations for time-to-value and continuous integration.

*Roll No: 1024030764; Email: agupta15_be24@thapar.edu
†Roll No: 1024030766; Email: agoyal2_be24@thapar.edu
‡Roll No: 1024030767; Email: nbansal3_be24@thapar.edu

2 Objectives

Primary goal. Build a campus web system that makes placement-round readiness and assignment submission outcomes visible, enforceable, and testable through a demonstrable Week 7 prototype.

Specific objectives.

1. Implement role-based authentication for Student, TA/Faculty, and Placement Admin, verified by login tests for each role.
2. By Week 7, deliver one end-to-end placement path: create company/round → document checklist → reminder log entry ($T-24$h).
3. By Week 7, deliver one end-to-end academic path: publish assignment with late policy → submit → automatic late flag → similarity stub score → TA grade.
4. Maintain 100% agreement between automated late flags and the stated policy on six fixed black-box boundary cases, within a twelve-test workflow and authorization suite.
5. Keep the similarity check as an explicit stub (not a research NLP system); run CI (build and tests) and publish project documentation via the course MkDocs Pages pipeline.

Figure 1 defines the actor-to-system boundary used to keep the prototype focused. Students use both deadline workflows, while TA/Faculty and Placement Admin users manage only the records authorized for their roles.

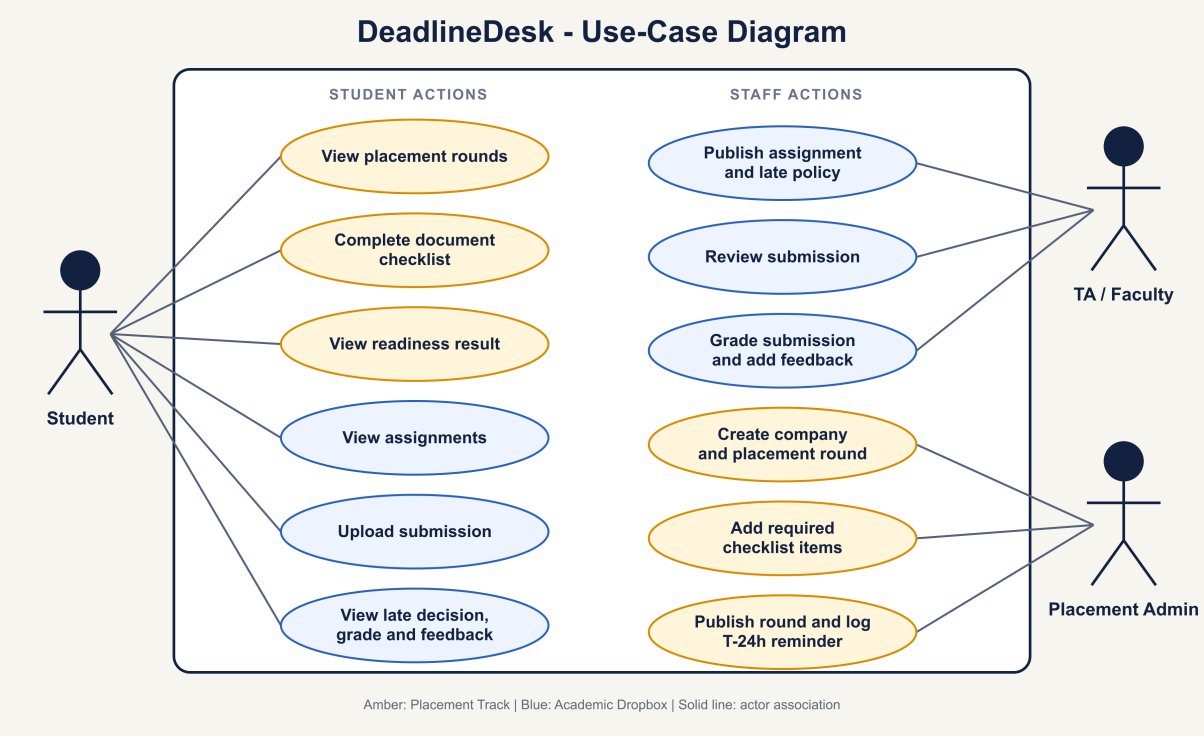

Figure 1: DeadlineDesk use-case diagram for the Week 7 scope.

3 Methodology

This is an engineering methodology: architecture, module development, integration, and validation. Figure 2 shows the system context; Figure 3 decomposes the system into the implemented processes and persistent stores.

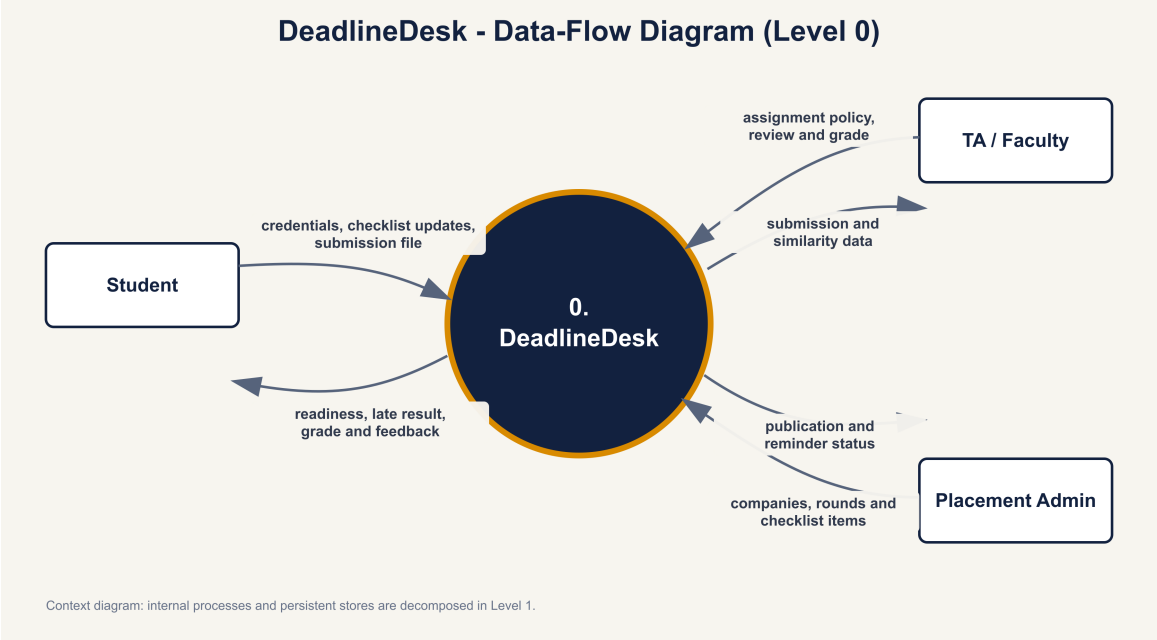

Figure 2: Level 0 data-flow diagram: external data exchanged with DeadlineDesk.

3.1 System architecture and technology stack

The prototype uses a layered Django web architecture:

- **Presentation:** responsive web UI for student, TA, and admin portals.
- **Application:** role-protected Django views and forms for authentication, CRUD operations, workflow transitions, and grading.
- **Domain:** service functions and ORM entities for users, roles, placement rounds, checklists, assignments, submissions, grades, reminders, and audit events.
- **Persistence:** SQLite and local media for the Week 7 prototype, isolated behind Django's ORM for a later managed-database migration.

The implemented stack is Django, server-rendered HTML/CSS, SQLite, Pytest, GitHub Actions, and MkDocs with GitHub Pages. The design relies on mature components rather than novel algorithms.

3.2 Module development

1. **Authentication and roles.** Django session authentication with stored roles and route guards.
2. **Placement Track.** An administrator publishes a company and round with open/close times. Round states: draft → published, with open and closed states derived from the configured timestamps. Students complete required checklist items; readiness requires all required items to be complete.
3. **Academic Dropbox.** A TA sets the due time and late policy (reject / accept-with-penalty / grace-window). Late evaluation is a function of submission time, due time, and policy. Accepted submissions move from submitted → under-review → graded; the late outcome is recorded separately.
4. **Reminders and audit.** Persist a T-24h reminder log and the placement/academic audit events. Email delivery is outside the Week 7 baseline.

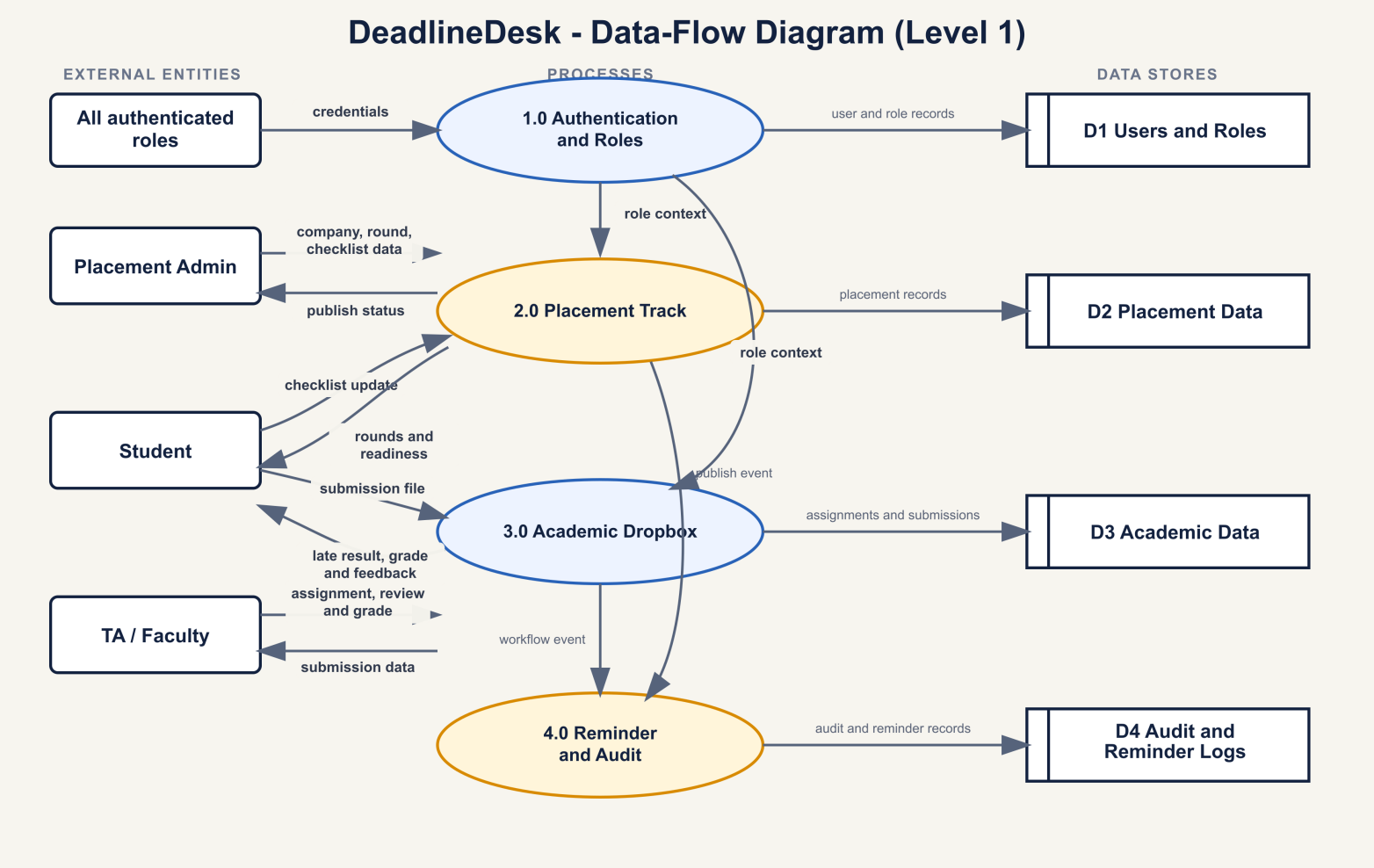

Figure 3: Level 1 data-flow diagram for implemented processes and persistent stores.

3.3 Integration, CI/CD, and validation

Work proceeds in weekly increments: build, test, and demonstrate thin features. CI runs migrations, Django system checks, and automated tests on each push; MkDocs documentation deploys through GitHub Pages. A deterministic seed command creates the three demo roles and representative deadline data.

Evaluation. Primary metric: on-time completion rate (OTCR)—the fraction of tracked deadlines where the required student action occurs before cutoff, taken from database timestamps. Prototype verification also requires full agreement of the late-policy decision table, authorization checks, valid state transitions, and repeatable end-to-end demonstrations. The Week 7 suite contains twelve passing automated tests.

Risks and mitigations. Two-module scope is controlled by a shared core and thin Week 7 paths. Reminder automation is demonstrated as a database log; email delivery is outside the baseline. The similarity component is labelled as a stub in the SRS. Official college email addresses are recorded on the team declaration sheet.

Scalability. Pagination, indexes, and clear module boundaries support growth in users and deadlines. Django ORM boundaries allow a managed-database migration without changing the two domain workflows.

4 Delivery Record Through Week 7

Week	Task	Deliverable
W3–W4	Problem definition, proposal, presentation, repository setup	Proposal, slides, project site
W5	Architecture, authentication, policies, user flows, and SRS	Documented shared core
W6	Placement Track, Academic Dropbox, grading, and responsive UI	Both thin paths integrated
W7	Automated tests, browser QA, CI, documentation, and demo seed	**Verified prototype (MST)**

5 Resources / Budget

Human resources.

- **Aryan Gupta (1024030764):** architecture, CI/CD, placement module.
- **Aksh Goyal (1024030766):** academic dropbox module and late-policy tests.
- **Naveen Bansal (1024030767):** user interface flows, documentation site, demonstration script.

Material resources and budget. Personal laptops, GitHub, SQLite, and LaTeX cover the prototype and documentation needs. No specialised hardware or external funding is required.

6 Summary

DeadlineDesk combines two deadline workflows with clear roles, explicit status machines, continuous integration, and measurable policy correctness. The Week 7 baseline delivers both end-to-end paths, twelve passing tests, documentation, and a repeatable role-based demonstration. Email delivery and plagiarism detection remain explicitly outside this baseline.